Building multi-tenant agents using Gemini Enterprise Agent Platform

Authors: Nithin Reddy Cheruku, Navneet Tuteja, Dhawal Patel

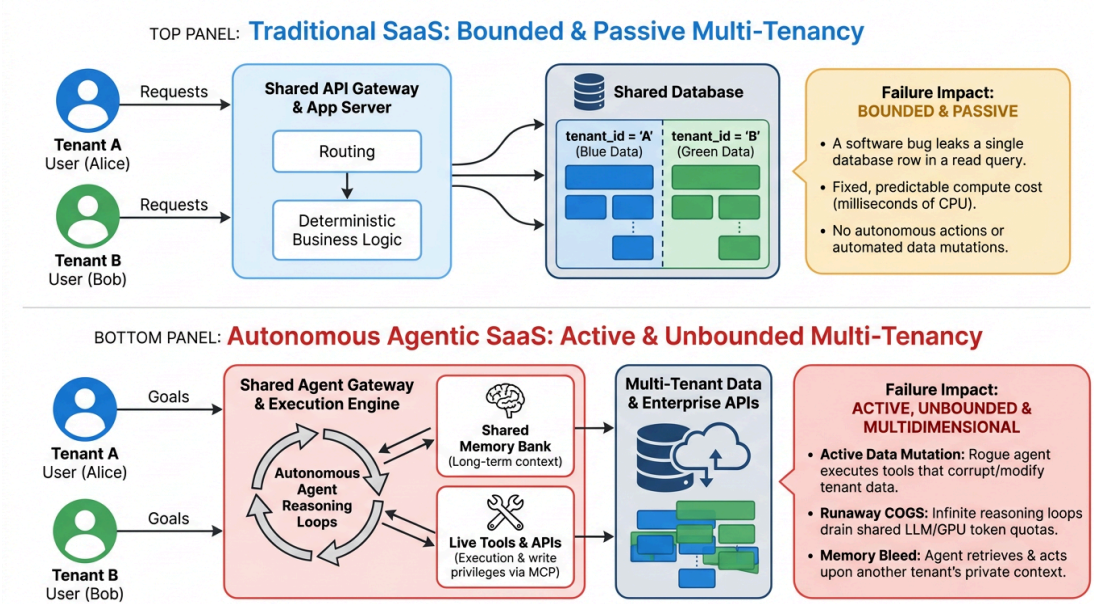
Overview

Independent software vendors are moving agents out of pilots and into products. The engineering question is no longer whether an agent can complete a task, but whether one platform can serve thousands of customers with agents without leaking data across accounts, eroding gross margin, or failing an enterprise security review.

That is a multi-tenancy problem, and it is not the multi-tenancy problem ISVs have already solved. In conventional SaaS, isolation is enforced at a small number of predictable choke points: an authenticated request, a scoped query, a row filter. The code path is deterministic, so it can be enumerated, tested, and demonstrated to an auditor. An agent removes that guarantee. It decides at runtime which tools to call, what to retrieve, how many turns to take, and which sub-agents to spawn. The execution path is generated rather than written, and every architectural assumption that depended on knowing the path in advance has to be re-established somewhere else.

This post sets out the components where that work happens, and how Gemini Enterprise Agent Platform (GEAP) implements each one.

Challenges



The first challenge is that the isolation boundary moves inward. Tenant identity can no longer be validated once at the edge and then assumed for the remainder of the request. It has to survive every hop the agent takes, through the planner, into each sub-agent, across every tool call and retrieval, and down to the model invocation itself. It also has to be immutable once set, because an agent that can rewrite its own tenant context is an agent with no tenancy at all. Isolation stops being a perimeter and becomes a property that each hop has to carry.

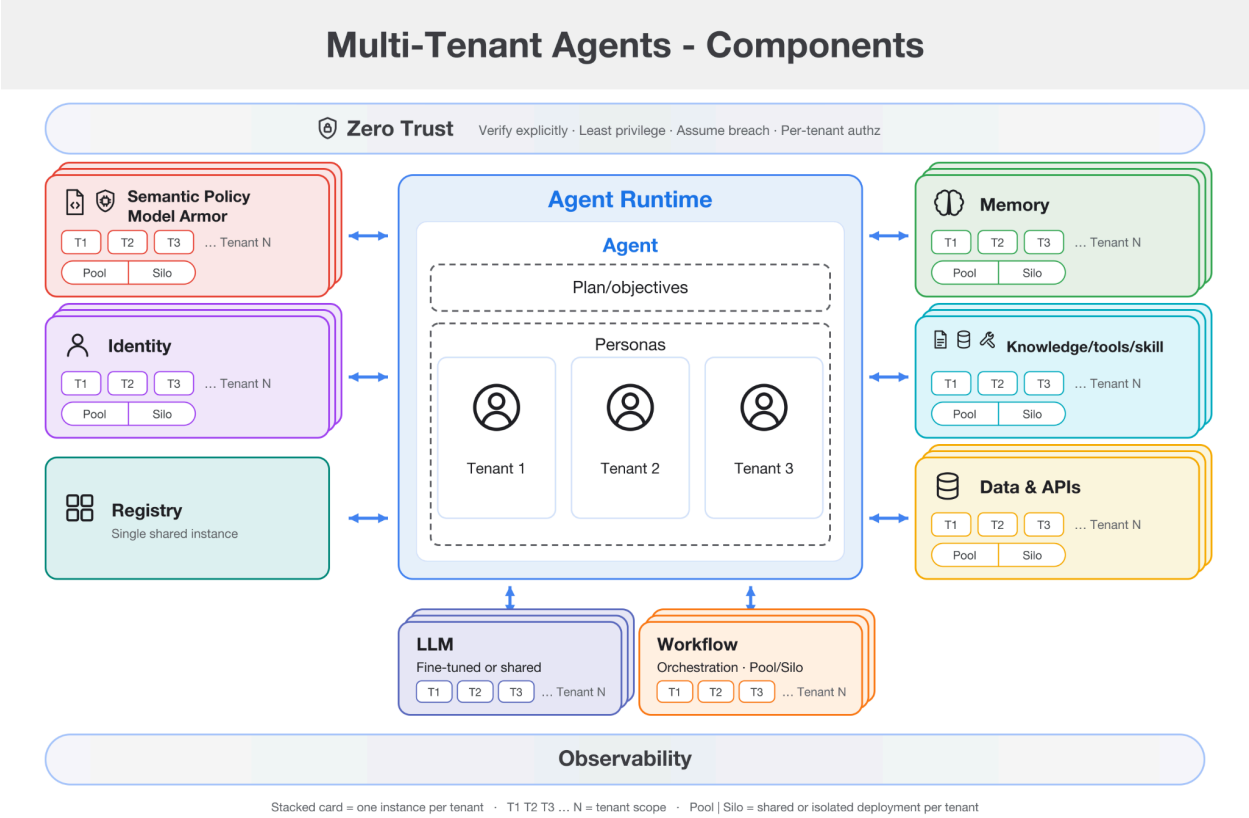
The untrusted input now reaches the control path. In deterministic software, user input is data that code acts upon. In an agentic system it is partly instruction, which makes the model a legitimate attack surface rather than a passive component. An agent compromised through its prompt is a confused deputy that already holds valid credentials and is already inside the boundary, so it does not need to escalate privilege to cause damage.

In addition, the cost becomes a runtime property rather than a planning assumption. Traditional SaaS enjoyed stable per-tenant infrastructure costs that could be modeled in a spreadsheet. Agentic workloads do not behave that way. Turn counts vary per request, recursive delegation multiplies calls, and retrieval expands context unpredictably, so one tenant can consume orders of magnitude more than another on an identical price plan.

The one that shapes the architecture most directly, is that isolation stops being a single decision. Memory, tools, data, model context and the runtime each carry their own

pooled-versus-siloed answer, and each answer carries different compliance obligations, blast radius and unit cost. There is no single multi-tenancy switch to flip. Every component in the stack independently answers one question: is this shared across tenants, or dedicated per tenant? Answering it well lets a platform serve thousands of self-serve tenants alongside a handful of regulated enterprises on one control plane. Answering it poorly means either paying for isolation nobody asked for, or discovering the gap during a customer security review.

The diagram below shows the ten components where that question has to be answered, and the two planes that cut across all of them.



Each component below covers what it does, the tenancy decision it forces, and the GEAP service that implements it. Implementation depth for each follows later in this post.

Foundational components for building multi-tenant agents

Identity

Agentic applications carry four distinct identities, and conflating them is the most common source of multi-tenant authorization bugs.

User Identity (ID-1 | Who is asking): The authenticated human principal initiating the session, verified and issued by the enterprise IdP (e.g., Cloud Identity, Entra ID, Okta).

Agent Identity (ID-2 | What is running): The platform-issued workload identity (e.g., SPIFFE, mTLS, API keys, Signed JWT, X.509 certificates) that uniquely identifies the running agent runtime and authorizes its access to infrastructure-level services (such as IAM policies, BigQuery, and Cloud Audit Logs).

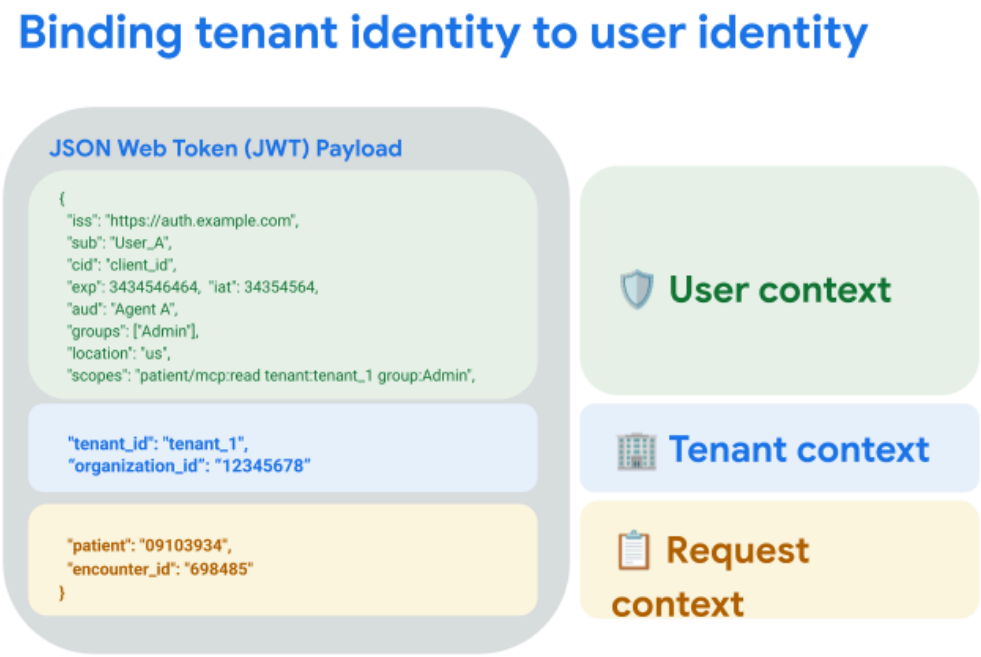
On-Behalf-Of / Delegated Identity (ID-3 | Whose permissions apply downstream): The user-delegated credential (e.g., OAuth 2.0 access token) brokered via explicit consent, allowing the agent to perform actions against external systems (such as Slack or Google Workspace) strictly within the user's personal permission envelope.

Tenant Identity (Optional | ID-4 | Which boundary governs isolation): The verified organizational boundary (tenant_id) that enforces data segregation, memory partitioning, telemetry routing, and row-level security across shared execution tiers.

Binding Tenant Identity to User Identity

To prevent spoofing and eliminate forgeable client-side signals, tenant identity is cryptographically bound to the user identity at ingress within a signed JSON Web Token (JWT). By embedding the validated tenant_id claim directly alongside the user principal (sub) and authorized scopes (tenant:tenant_1), the platform produces an immutable security context. Downstream planners, sub-agents, memory stores, and database RLS engines inherit this unified token across every execution hop without relying on ambient parameters. However, In mature enterprise environments with existing multi-tenant services, upstream microservices often generate and propagate tenant context dynamically in-flight via custom HTTP headers (e.g., X-Tenant-ID, Tenant-ID, or path prefixes). To safely support these workflows without introducing confused-deputy risks, make sure the Ingress proxies / Agent Gateways intercept in-flight headers, validate them against the user's authenticated directory entitlements

or tenant catalog (e.g., checking if `User_A` belongs to `Tenant_1`), and strip untrusted raw headers and the gateway mints or enriches the downstream signed JWT/intent token with the verified tenant claim before dispatching to the agent runtime, bridging legacy header-driven routing with zero-trust cryptographic enforcement.

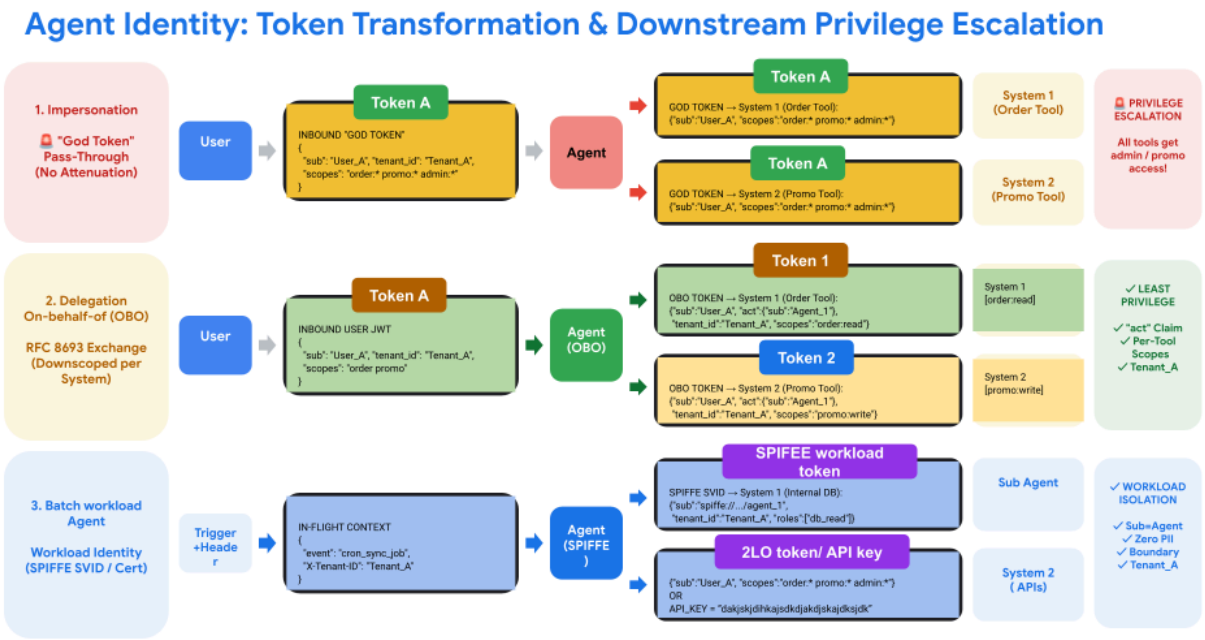


Gemini Enterprise Agent Platform treats all three as first-class primitives rather than application concerns. Agent Identity issues every agent a unique, lifecycle-bound SPIFFE ID backed by an auto-provisioned X.509 certificate, and that ID acts directly as an IAM principal — usable in Principal Access Boundary policies and VPC Service Controls, and recorded against every call in Cloud Audit Logs. Unlike service accounts, these identities cannot be impersonated, issue no long-lived keys, and retire with the agent runtime, which removes the orphaned-credential problem that shared service accounts create at scale. Access tokens are cryptographically bound to the certificate through mTLS and DPoP so a stolen token cannot be replayed outside its authorized runtime. For delegated access, the Agent Identity auth manager brokers the three-legged OAuth flow, manages consent, and decrypts credentials inside the Agent

Gateway so the agent never handles a raw secret. For an ISV, the practical effect is that the tenant boundary is enforced by platform-issued cryptographic identity rather than by application code remembering to pass a tenant ID.

2. Identity Propagation and tenant isolation

When an AI agent interacts with downstream tools and data custodians, how identity and tenant context are propagated determines the system's security posture. Enterprise agent architectures typically encounter three propagation patterns:



1. **Impersonation (Raw "God Token" Pass-Through):** The user's inbound token (`Token A`) containing broad permissions (`order:*`, `promo:*`, `admin:*`) is forwarded verbatim by the agent to every downstream tool. Because there is no attenuation or actor attribution, every tool receives ambient, over-privileged access, introducing critical **Privilege Escalation** and confused-deputy risks if the agent prompt or a tool is compromised.
2. **Delegation / On-Behalf-Of (RFC 8693 Token Exchange):** The agent exchanges the user's inbound token for individual, downscoped tokens tailored per target system (`Token 1` with `order:read` for the Order Tool; `Token 2` with `promo:write` for the Promotions Tool). By embedding an **actor claim** (`"act": {"sub": "Agent_1"}`) alongside the user subject and tenant boundary

(`tenant_id: "Tenant_A"`), this pattern enforces **Least Privilege** and ensures end-to-end auditability.

3. **Batch / Autonomous Workload (SPIFFE Workload & Vault Exchange):** For non-human or scheduled tasks initiated by background triggers, the agent runs under its own cryptographic workload identity (e.g., **SPIFFE SVID**). To interact downstream, the agent either presents its SPIFFE token directly to service-mesh-aware systems or leverages a Credential Vault to exchange it for **tenant-scoped 2LO OAuth tokens or API keys** for legacy/3P APIs—maintaining strict **Workload Isolation** with zero user PII exposure.

Agent Deployment Models

Architecting multi-tenant agent platforms forces ISVs to navigate the **Agentic Trilemma**—the inherent architectural tension between **Tenant Isolation**, **Cost Efficiency**, and **Operational Simplicity**. The **Silo Model** dedicates separate infrastructure (projects, runtimes, and memory) to each tenant, delivering zero blast radius and effortless compliance at the expense of high idle costs and fleet sprawl. The **Pool Model** runs all tenants across a shared runtime fleet, maximizing compute density and simplifying operations through logical JWT/RLS partitioning, but introducing noisy-neighbor and confused-deputy risks. To resolve this tension, most enterprise ISVs adopt a **Hybrid Model**, routing high-volume standard tiers to a cost-effective shared pool while dynamically dispatching premium or regulated enterprise workloads to dedicated, physically isolated silo perimeters.

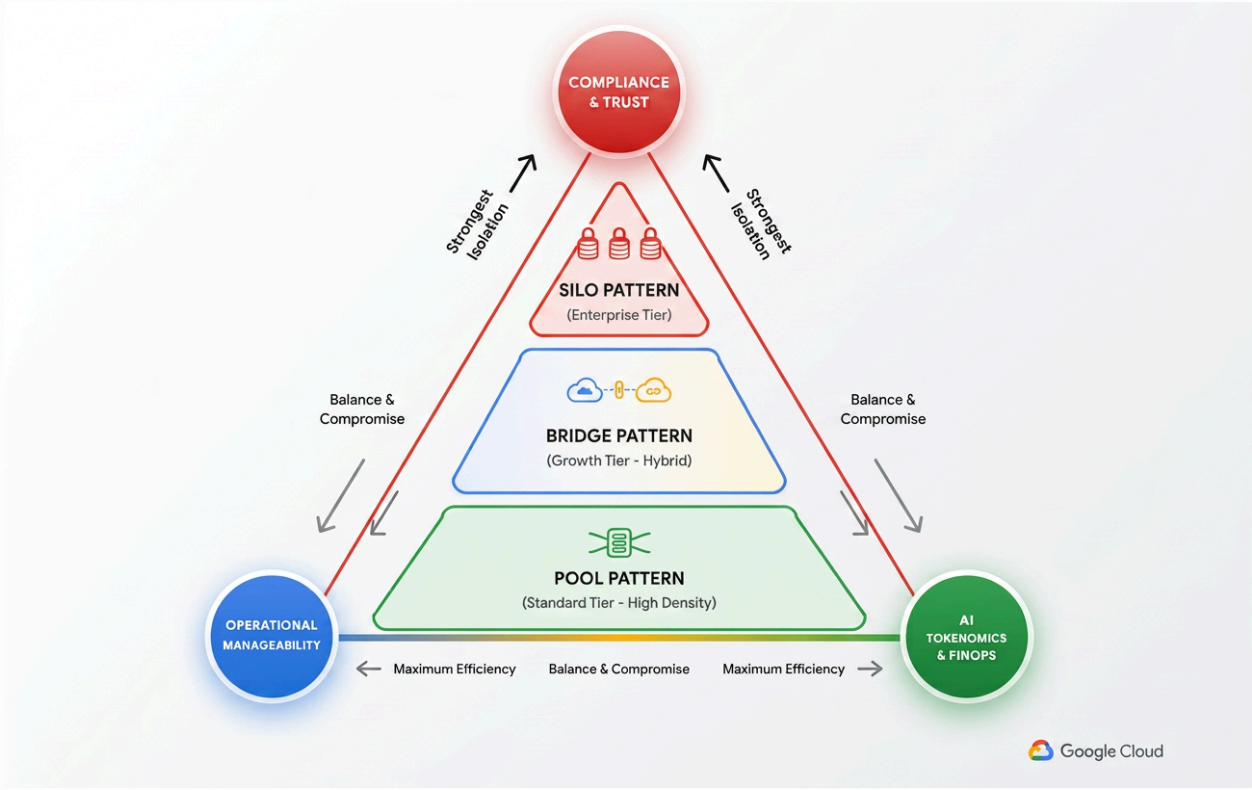


Figure 1: Balancing compliance, manageability, and SaaS gross margins.

4. Architectural Foundation: Overview of Gemini Enterprise Agent Platform (GEAP)

The Gemini Enterprise Agent Platform (GEAP) is Google Cloud's unified, enterprise-grade platform designed to build, scale, govern, and optimize autonomous agents. For multi-tenant SaaS ISVs, GEAP functions as a managed control and execution plane over existing (brownfield) cloud infrastructure.



Figure 2: Gemini Enterprise Agent Platform (GEAP) Architectural Pillars

Why B2B ISVs Build Multi-Tenancy on GEAP

- Build (Open & Framework-Agnostic): Supports ADK 2.0 alongside third-party (3P) agent frameworks (LangGraph, CrewAI, AutoGen). Integrates frontier Gemini models and open models via Model Garden, coupled with open standard protocols (A2A, Stateless MCP, A2UI, and enterprise connectors).
- Scale (Managed Compute & State Isolation): Agent Runtime provisions sub-second warm-start execution with Agent Sandbox backed by gVisor microVMs. Agent Sessions and Agent Memory Bank provide managed multi-tenant hierarchical memory with tenant-isolated Customer-Managed Encryption Key (CMEK) partition boundaries.
- Govern (End-to-End Enterprise Perimeter Security): Agent Gateway (PEP) and Agent Registry (PDP) enforce tenant-scoped agent authorization and dynamic tool entitlements. Vertex AI Model Armor protects against prompt injection and Data Loss Prevention (DLP) leaks, while Agent Identity (SPIFFE) and Agent Policy enforce Natural Language Constraints (NLCs) and anomaly detection.
- Optimize (Continuous Quality & Tokenomics): Agent Evaluation (LLM-as-a-judge scoring), Agent Simulation, and Agent Observability feed telemetry directly into BigQuery Agent Analytics, providing granular per-tenant COGS attribution and automated agent tuning.

6. Deep Dive: The Three Core Tenancy Topologies

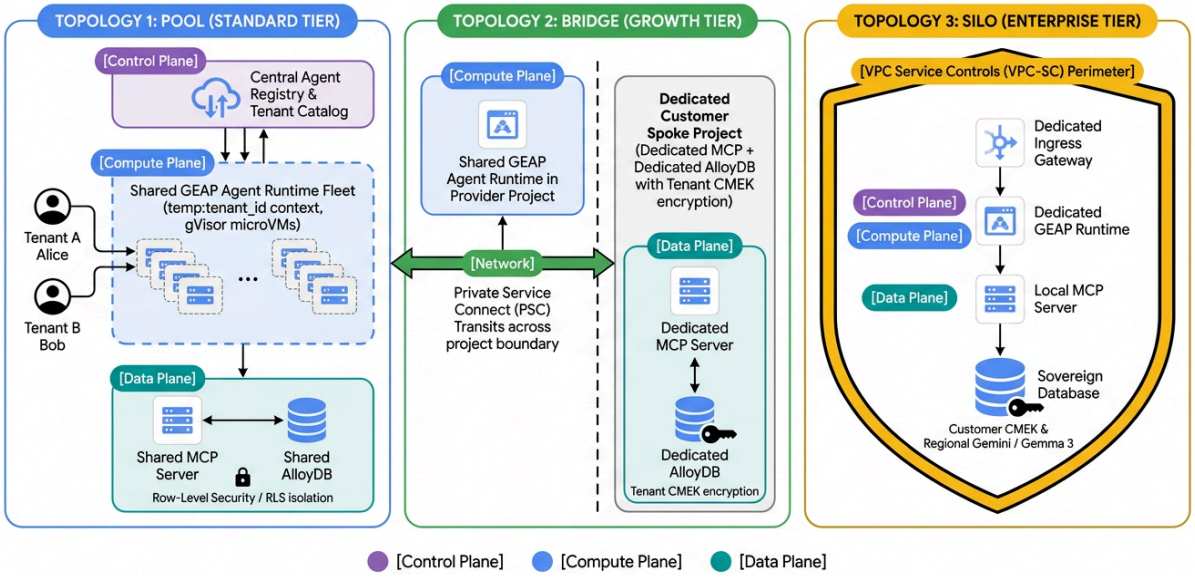
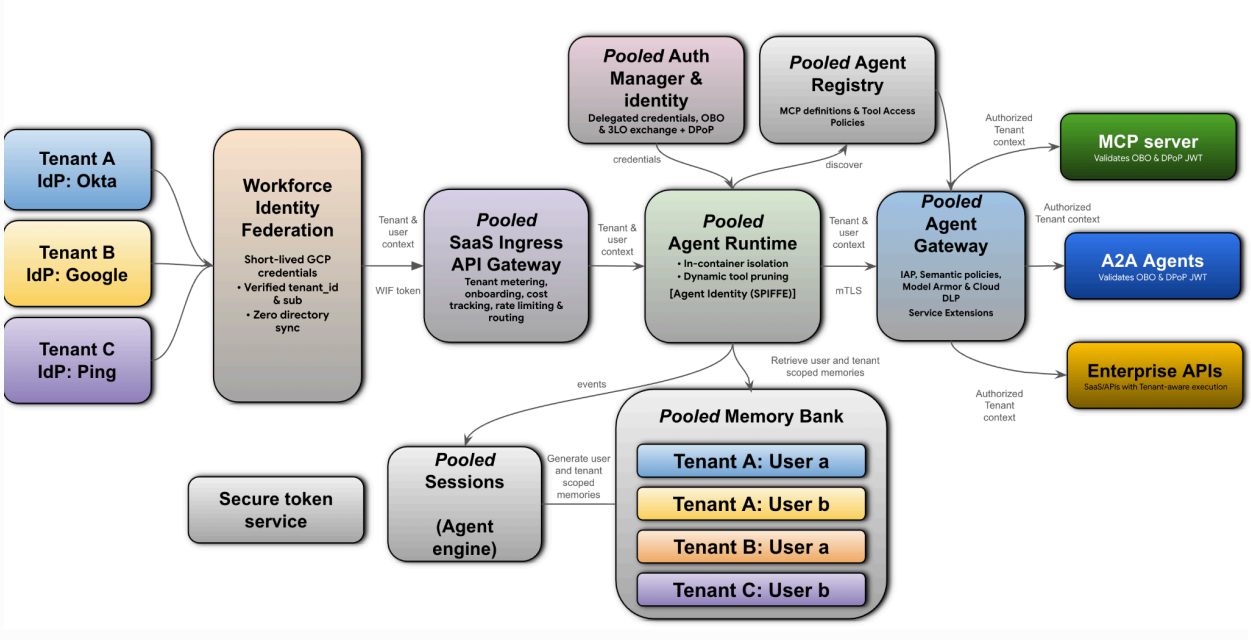


Figure 4: The Three Multi-Tenant Tenancy Topologies (Pool, Bridge, Silo) with Control, Compute, and Data Planes

Pattern 1: High-Density Pooled Architecture (Standard Tier)

In the **High-Density Pooled Architecture**, standard-tier SaaS tenants share a unified, highly scalable Google Cloud infrastructure across the control, compute, and data planes. Tenant isolation is enforced end-to-end through **cryptographic tokens, request-scoped in-container boundaries, exact-match memory scoping, and policy-driven egress gating**—ensuring zero cross-tenant data leakage while maximizing resource efficiency.



End-to-End flow

1. Inbound Authentication & Identity Federation (Tenant Realms)

- **Corporate SSO:** Users from different enterprise realms (e.g., Tenant A with Okta, Tenant B with Google Identity, Tenant C with Ping) authenticate directly against their native Enterprise IdPs.
- **Workforce Identity Federation (WIF):** The corporate SAML 2.0 / OIDC assertions federate into Google Cloud via Security Token Service (STS). WIF mints short-lived federated credentials with cryptographically verified token claims (`tenant_id`, `sub`/`user_id`, `roles`) without requiring cross-organization directory sync.

2. Perimeter SaaS Ingress & Policy Enforcement (Ingress Gateway)

- **Pooled SaaS Ingress API Gateway:** Acts as the shared edge entry point handling tenant onboarding, metering, cost attribution, rate limiting, and tenant-aware routing.
- **Token Validation & Header Propagation:** The gateway validates the inbound token signature against IdP/WIF JWKS, extracts verified tenant context, and forwards an immutable **`Tenant & user context in JWT`** downstream.

3. Execution, Orchestration & Memory Partitioning (Agent Runtime)

- **In-Container Turn Isolation:** The **Pooled Agent Runtime** executes turns inside isolated, request-scoped container contexts in the Google ADK fleet.
- **Dynamic Tool Pruning:** Before invoking the model, the runtime excises unauthorized tools from the request schema based on tenant entitlement policies.
- **Schema Discovery & Attestation:** The runtime queries the **Pooled Agent Registry** to discover and attest JSON-RPC tool schemas and MCP definitions.
- **Memory Bank Exact-Match Scoping:** Long-term memory retrieval and event consolidation in **Memory Bank (Agent Engine)** enforce strict exact dictionary matching using Memory scope={"tenant_id", "user_id"}
- **Session Persistence:** Session turns are written to **Pooled Sessions (Agent Engine)** keyed by composite composite identifiers (`{tenant_id}:{user_id}:{session_id}`).

4. Credential Delegation & Identity Exchange (Auth Manager)

- **Pooled Auth Manager & Identity:** Holds delegated user credentials and OAuth refresh tokens.
- **OBO & 3LO Delegation:** Exchanges the inbound user token for downscoped, tool-specific short-lived tokens bound with **RFC 9449 DPoP** proofs for On-Behalf-Of (OBO) execution.

5. Egress Authorization, Safety & Tool Execution (Egress Gateway)

- **Pooled Egress Agent Gateway (PEP Proxy):**
    - **Policy Resolution:** Queries the **Pooled Agent Registry** to resolve tenant-specific tool access policies.
    - **Service Extensions:** Executes custom WebAssembly/gRPC callouts for fine-grained access control (FGAC) governing which tenant can invoke specific tools.
    - **Identity-Aware Proxy (IAP) & Semantic Policies:** Enforces IAM and semantic guardrails.
    - **Content Safety:** Runs **Model Armor & Cloud DLP** to filter prompt injections, screen payloads, and redact sensitive PII/SSNs.
- **Authorized Downstream Execution:**
    - **Pooled MCP Server:** Validates OBO JWTs and DPoP proofs before executing tool microservices.
    - **A2A Multi-Agent Collaboration:** Dispatches delegated sub-tasks while strictly propagating tenant context across agent boundaries.
    - **Pooled Enterprise APIs & Databases:** Calls IAM-protected downstream services and databases with Row-Level Security

Pattern 2: Decoupled Compute & Dedicated Data Spokes (Bridge Pattern — Growth Tier)

The Bridge Pattern represents a strategic middle ground between full pooling and dedicated silos. Instead of forcing a binary architectural choice, the Bridge pattern allows SaaS architects to decouple tenancy at different layers of the stack based on commercial tiering and compliance requirements.

Pattern 2A: Tier-Based Hybrid Bridge (Silo for Enterprise, Pool for Standard)

In this variation, a single SaaS platform operates coexisting execution topologies governed by a unified SaaS Control Plane and Ingress Dispatcher. High-compliance enterprise customers run in dedicated Siloed projects, while standard self-serve customers execute on a High-Density Pooled runtime fleet.

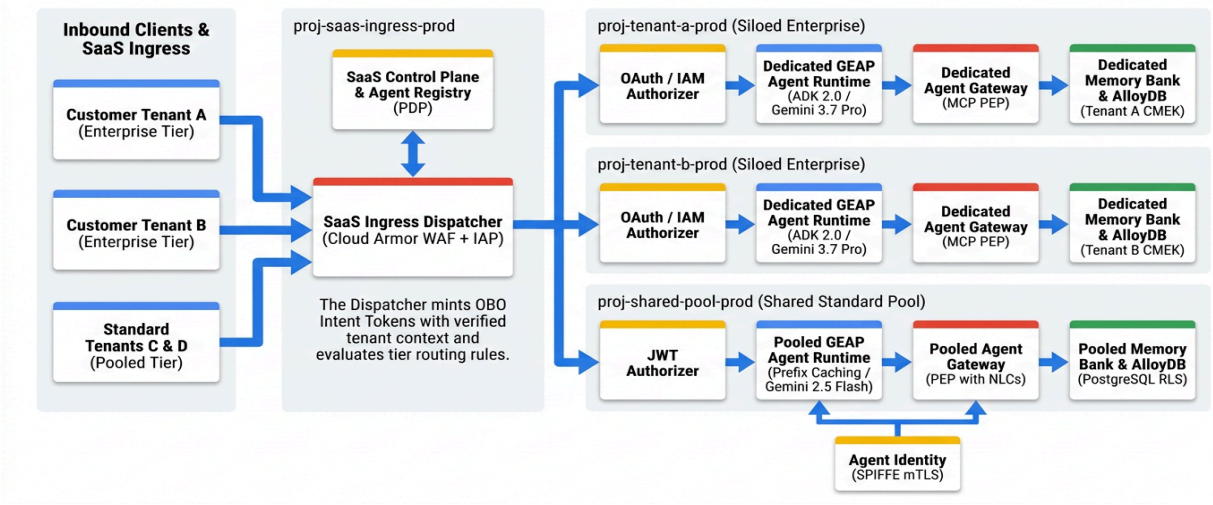


Figure 6A: Pattern 2A: Tier-Based Hybrid Bridge Architecture (GEAP)

Best For: SaaS ISVs scaling hundreds of standard self-serve tenants while simultaneously onboarding large enterprise accounts that require isolated compute and memory guarantees.

Pattern 2B: Layer-Decoupled Hybrid Bridge (Shared Compute + Dedicated Data Spokes via PSC)

In this variation, compute and reasoning loops run in a shared provider runtime fleet, while tools, memory banks, and datastores are deployed into dedicated, tenant-isolated GCP spoke projects connected via Private Service Connect (PSC)

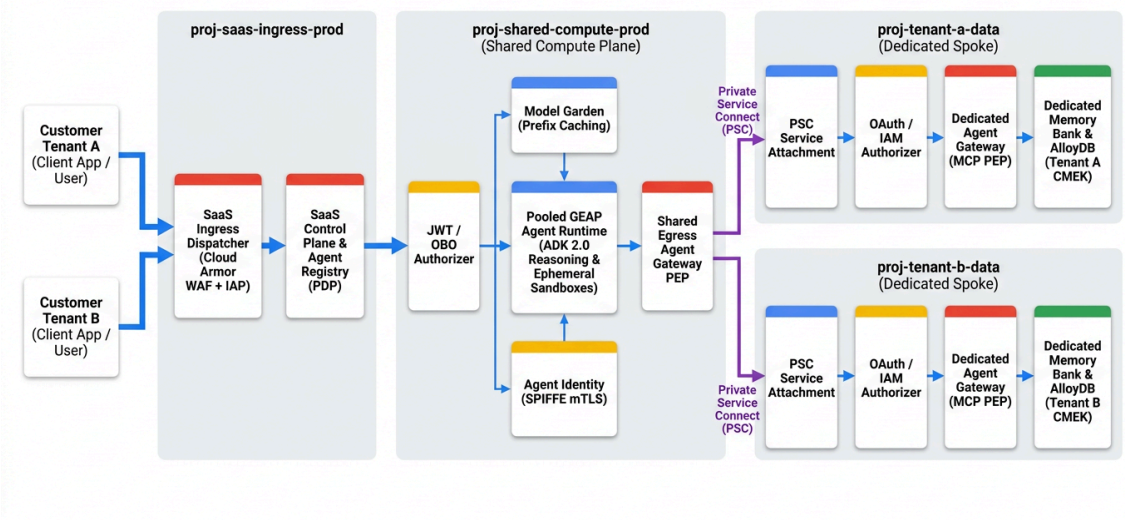


Figure 6B: Pattern 2B: Layer-Decoupled Hybrid Bridge Architecture (GEAP)

Best For: Regulated workflows (such as Security Operations Center / SOC Analyst agents or Fintech backends) where reasoning compute is stateless and shared, but tools (e.g., customer firewall APIs, internal email

connectors) and sensitive databases must remain strictly isolated inside customer-dedicated VPC spoke perimeters with Customer-Managed Encryption Keys (CMEK).

How GEAP Implements the 7 Subsystems in Pattern 2 (Bridge)

1. Inbound Identity: The SaaS Ingress Dispatcher in proj-saas-ingress-prod (powered by Cloud Armor Web Application Firewall - WAF and IAP) validates inbound OIDC tokens, consults the SaaS Control Plane PDP for subscription tier entitlements, and routes traffic (2A: routes Enterprise to siloed projects, Standard to pooled project; 2B: terminates at shared ingress).
2. Ingress Agent Gateway (PEP): In 2A, dedicated GEAP Ingress Gateways run in each silo while a shared Ingress Gateway serves the pool; in 2B, a shared Ingress Gateway executes Model Armor prompt screening and mints RFC 9449 DPoP intent tokens for the shared runtime.
3. Agent Runtime & Orchestration:
    - Pattern 2A: Dedicated GEAP Agent Runtime instances in proj-tenant-a-prod running ADK 2.0 with dedicated concurrency quotas; shared runtime fleet in proj-shared-pool-prod for standard tenants.
    - Pattern 2B: Shared GEAP Agent Runtime in proj-shared-compute-prod executing reasoning loops in ephemeral sandboxes (gVisor), with custom tenant skills dynamically mounted via the Managed Agents API.
4. State & Multi-Level Memory:
    - Pattern 2A: Dedicated GEAP Memory Bank instance per enterprise project with Customer CMEK; logically partitioned Memory Bank for the pooled tier.
    - Pattern 2B: Ephemeral turn session store in shared compute; long-term episodic/semantic Memory Bank deployed inside dedicated customer spoke projects (proj-tenant-a-data) encrypted with Customer CMEK.
5. Foundation Models & Caching: Tier-differentiated model routing via Model Garden—Gemini 3.7 Pro for enterprise tiers with guaranteed Provisioned Throughput (PTU); Gemini 2.5/3.7 Flash with shared prompt prefix Context Caching for standard/growth tiers.
6. Egress Gateway & Guardrails:
    - Pattern 2A: Dedicated Egress Gateway evaluating custom enterprise Natural Language Constraints (NLCs) inside each silo project.
    - Pattern 2B: Shared Egress Gateway in proj-shared-compute-prod evaluating tenant-customized Natural Language Constraints and routing tool calls across dedicated Private Service Connect (PSC) forwarding rules directly to spoke projects.
7. Governed Data & Dynamic Tool Fabric:
    - Pattern 2A: Dedicated MCP Tool Servers and CMEK AlloyDB / BigQuery in enterprise silos; shared MCP and database RLS in the pooled project.
    - Pattern 2B: Dedicated Stateless MCP Tool Servers and AlloyDB datastores deployed in customer spoke projects (proj-tenant-a-data), exposed securely via PSC Service Attachments with zero public internet exposure.

Pattern 3: Sovereign Multi-Tenant Silo Architecture (Enterprise Tier)

For regulated enterprise clients (financial services, healthcare, defense) requiring strict regulatory compliance (HIPAA, FedRAMP High, PCI-DSS), the Silo Pattern deploys completely dedicated infrastructure within a dedicated Google Cloud project wrapped in a VPC Service Controls (VPC-SC) security perimeter.

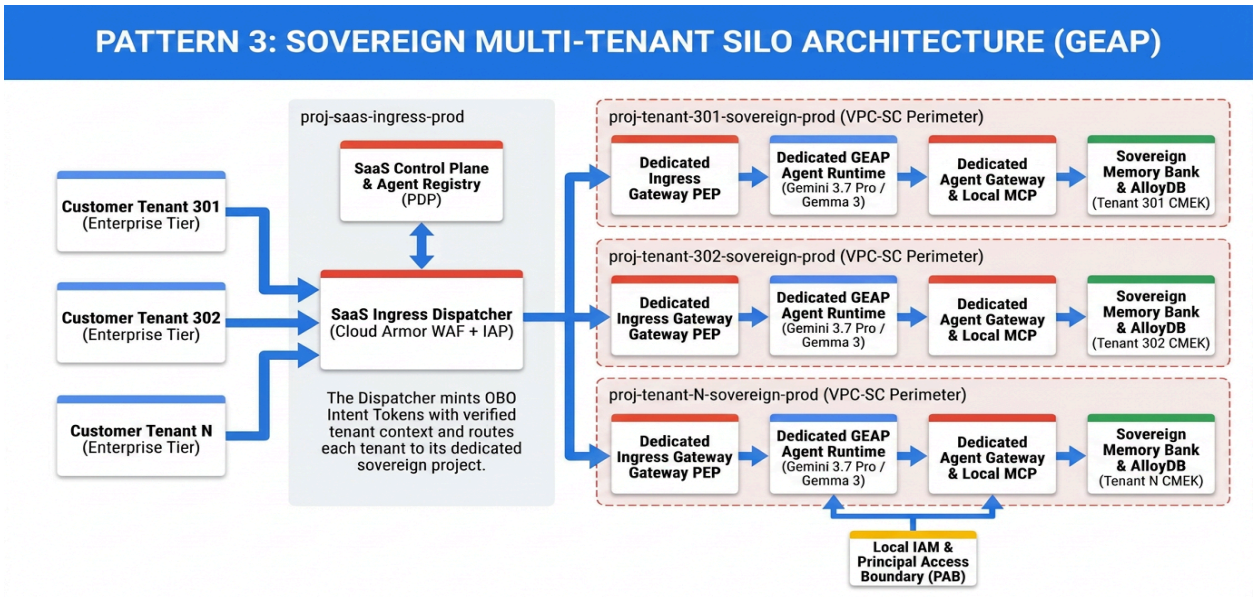


Figure 7: Pattern 3: Sovereign Multi-Tenant Silo Architecture (Enterprise Tier)

How GEAP Implements the 7 Subsystems in Pattern 3 (Silo)

1. Inbound Identity: Dedicated enterprise IdP federation (SAML 2.0 / OIDC direct integration with customer Okta/Entra ID), client mutual TLS (mTLS) certificate validation, and Principal Access Boundaries (PAB) restricting IAM role assumption strictly to the sovereign project perimeter.
2. Ingress Agent Gateway (PEP): Dedicated GEAP Agent Gateway PEP deployed inside the tenant project perimeter. Enforces zero shared ingress infrastructure, validates tenant mTLS, and runs isolated Model Armor screening policies.
3. Agent Runtime & Orchestration: Dedicated GEAP Agent Runtime running in proj-tenant-301-sovereign-prod enclosed in a VPC Service Controls (VPC-SC) perimeter. Executes ADK 2.0 agents with dedicated compute instances and dedicated Agent Sandboxes (gVisor) for isolated code/tool execution.
4. State & Multi-Level Memory: Dedicated GEAP Memory Bank & GEAP Agent Sessions deployed entirely inside the tenant's VPC-SC project perimeter, encrypted exclusively with customer-managed encryption keys (Tenant CMEK).
5. Foundation Models & Caching: Sovereign model execution options:
    - Self-hosted open-weight Gemma 3 (1B/4B/12B/27B) served via vLLM on GKE deployed directly inside the VPC-SC perimeter (fully air-gapped from external networks).
    - Regional Gemini 3.7 Pro endpoints accessed via VPC-SC under legally binding zero-data-retention agreements.

6. Egress Gateway & Guardrails: Dedicated in-perimeter GEAP Egress Agent Gateway PEP evaluating strict enterprise Natural Language Constraints (NLCs) and DLP inspection. Physical network egress to unapproved Google Cloud projects or internet endpoints is blocked at the Google network layer by VPC-SC.
7. Governed Data & Dynamic Tool Fabric: 100% Dedicated Stateless MCP Servers connecting to dedicated sovereign AlloyDB / BigQuery instances with physical perimeter isolation, full CMEK encryption, and local database IAM policies.

What's Coming in Part 2

Multi-tenancy in the agentic era cannot be treated as an afterthought or bolted on at the API layer. It must be designed into the foundational control, compute, and data planes of your platform. In Part 2 of this series, Building the High-Density Pooled Tier: An End-to-End Implementation Blueprint, we trace a live agentic transaction (Alice from Tenant A analyzing Q3 financial variances) and deconstruct the end-to-end architecture and production code across all 7 subsystems in the pooled pattern.

Building the Pool Tier - An End-to-End GEAP Implementation

This section traces a single agentic turn end-to-end: Alice, a financial analyst at Tenant A, asks her platform's revenue agent to analyze Q3 data. We follow her request through identity verification, registry authorization, ephemeral execution, governed data access, and cost-controlled model inference all within the Pool tier.

The Pool tier does not rely on network segmentation or VM isolation. It relies on cryptographic identity, immutable context propagation, and fail-closed data policies. Every section is a hop across a boundary; every hop has an enforcement point and a failure mode.

Ingress

What happens: Alice from Tenant A submits "Analyze Q3 revenue." IAP verifies OIDC JWT signature, strips forged headers (X-Tenant-ID), and binds her enterprise identity. The Agent Gateway (The Hinge) validates tenant claims, exchanges her credential for a short-lived On-Behalf-Of (OBO) Intent Token, and applies inline Model Armor prompt filters.

Architectural depth:

Why OBO matters: The original JWT never leaves the ingress boundary. Downstream services act on behalf of Alice using a scoped, short-lived token. If exfiltrated, it expires before damage spreads.

Model Armor as fail-closed: If prompt injection is detected, the turn is rejected before compute is provisioned. This protects the ephemeral runtime from malicious warm-start costs.

Well-Architected hook: Security — defense in depth at the outermost boundary.

What breaks at 2 AM: Expired JWT clock skew, forged X-Tenant-ID headers from misconfigured proxies, Model Armor false positives on legitimate financial jargon.

Compute Plane — Registry, Tiering & Context Injection

What happens: The Gateway queries the Agent Registry to confirm finance-agent-v2 is authorized for tenant_a's subscription tier. The root orchestrator injects temp:tenant_id = "tenant_a" into the ADK InvocationContext. All child sub-agents spawned during multi-agent reasoning inherit this boundary via async contextvars over mTLS.

Architectural depth:

Registry as circuit breaker: If the agent assembly is not tier-authorized, the request is rejected before runtime provisioning. This prevents shadow AI and unvetted MCP tool exposure.

Immutability contract: temp:tenant_id is set once at the root and inherited. Sub-agents cannot override it. This is the keystone pattern — if this context leaks or is mutable, every downstream policy is useless.

Code snippet:

```
Python


# All sub-agents inherit via ADK Coordinator + A2A mTLS delegation
```

Well-Architected hook: Security (ABAC at every hop), Reliability (Registry rejection prevents invalid agent execution).

What breaks at 2 AM: Async contextvar leakage between concurrent sub-agents, Registry cache staleness after tier downgrade, A2A delegation chain expiration mid-turn.

Compute Plane — Ephemeral Runtime, Session, Memory & Caching

What happens: GEAP Agent Runtime provisions an ephemeral sandbox with strict CPU/RAM limits. The Session Store fetches Alice's chat history via hard key-prefixed lookups (tenant_a:alice_123). The Memory Bank retrieves cross-session knowledge under the same prefix. Gemini calls use shared base prompt context caching, while tenant-specific turns are stored under namespaced cache keys (tenant_a:cache_key). After the turn completes, the sandbox is destroyed.

Architectural depth:

Ephemeral as the isolation primitive: There is no persistent compute to patch, no VM sprawl, and no cross-tenant memory residue between turns. Isolation is enforced by destruction, not configuration.

Memory Bank namespace hygiene: Hard key-prefixing (tenant_id:user_id) prevents accidental global memory retrieval. TTL policies and CMEK at rest are mandatory for enterprise tiers.

Context caching economics: Shared base prompts (system instructions, tool schemas) are cached once and reused across all tenants. Only tenant-specific variables (financial metrics, user preferences) bypass the cache. This is where 80% Pool margins come from.

Well-Architected hook: Performance (warm-start, cache hit ratios), Reliability (resource limits prevent runaway loops), Cost Optimization (shared prefixes amortize inference cost).

What breaks at 2 AM: Memory key collision from missing prefix, cache poisoning via adversarial prompt injection, sandbox resource exhaustion from recursive sub-agent spawning.

Data Plane — The Generic 3-Hop Governed Access Pattern

Note: While this example uses AlloyDB PostgreSQL RLS, the same three-hop pattern applies to Cloud SQL, Spanner fine-grained access control, Firestore path-scoped queries, or third-party datastores — the invariant is tenant_id propagation from application session to data-layer policy enforcement

What happens: The agent executes data retrieval across three mandatory hops: (1) MCP Tool Layer invokes query_financial_db using the OBO Intent Token; (2) Knowledge Layer translates natural language into scoped SQL; (3) Persistence Tier sets SET LOCAL app.current_tenant_id = 'tenant_a' before querying AlloyDB. PostgreSQL RLS guarantees only tenant_a rows are returned.

Architectural depth — make it generic:

"While this example uses AlloyDB RLS, the 3-hop pattern is substrate-agnostic. The invariant is: tenant claim propagation from application session → data-layer policy enforcement. ISVs running Cloud SQL, Spanner, Firestore, or third-party stores (Snowflake, MongoDB Atlas) implement the same semantic: a session-scoped tenant identifier that the query engine evaluates before returning rows."

Reference implementations:

PostgreSQL/AlloyDB/Cloud SQL: SET LOCAL + RLS policies

Spanner: IAM fine-grained access control with request.tag tenant conditions

Firestore: Collection path isolation (/tenants/{tenant_id}/documents/)

BigQuery: Row Access Policies with FILTER USING (tenant_id = SESSION_USER())

Code snippet (PostgreSQL reference):

```sql
-- Hop 3: Persistence tier enforces tenant isolation
SET LOCAL app.current_tenant_id = 'tenant_a';

CREATE POLICY tenant_isolation ON financial_records
  USING (tenant_id = current_setting('app.current_tenant_id')::TEXT);
```

Well-Architected hook: Security (RLS is fail-closed — if tenant_id is unset, zero rows return), Cost Optimization (shared-schema density vs. dedicated-store overhead).

What breaks at 2 AM: Missing SET LOCAL in connection pool reuse, RLS bypass via superuser connections, NL-to-SQL injection that appends OR 1=1 before the RLS predicate is applied.

Egress & Observability — Tokenomics, Audit, and the Operational Contract

What happens: Before model execution, the Model Gateway verifies tenant_a's remaining token bucket quota. During execution, OpenTelemetry emits trace spans tagged with tenant_id: tenant_a, automatically redacting sensitive financial payloads at the gateway while logging the decision tree for CISO compliance. The synthesized response returns to Alice; the ephemeral sandbox is destroyed.

Architectural depth:

Token bucket as bulkhead: Not just billing — it is reliability. If Tenant B's agent enters an infinite reasoning loop, Tenant A's quota is untouched. Redis-backed token counters with per-tenant TTL prevent cardinality explosion.

Observability schema: Every span carries tenant_id, agent_id, tool_name, and model_id. Cloud Trace → BigQuery pipeline feeds per-tenant cost attribution and LLM-as-a-Judge evaluation datasets.

The operational contract: Pool tier runs on a single runbook because there is no infrastructure to patch. Upgrade velocity is global — new model versions roll out to all tenants simultaneously. The SRE team operates one fleet, not one hundred snowflakes.

Well-Architected hook: Cost Optimization (token budgets prevent margin erosion), Operational Excellence (tenant-scoped traces for incident response), Sustainability (Flash vs. Pro routing based on task complexity).

What breaks at 2 AM: Token bucket desync between Redis and BigQuery, trace cardinality explosion from high-cardinality user_id tags, billing export lag causing quota miscalculation.

Why the Pool Tier Is the Foundation

"The Pool tier is not 'shared infrastructure with fingers crossed.' It is a cryptographically governed, ephemeral execution model where isolation is enforced by identity, RLS, and sandbox destruction not by VM boundaries. You graduate to Bridge or Silo when contractual liability justifies the operational tax. Until then, the Pool tier is how ISVs ship agentic AI to thousands of tenants with one runbook, one SLO, and margins that survive."

The Pool tier is not "shared infrastructure with fingers crossed." It is a Policy Decision Point (PDP) in the Control Plane, a Policy Enforcement Point (PEP) in the Compute Plane, and a fail-closed data boundary in the Data Plane. We follow Alice from Tenant A as she asks her revenue agent to analyze Q3 data — tracing how her request crosses each plane, where decisions are made, where they are enforced, and what happens when things break at 2 AM.

Section 1: Control Plane — The Brain, Billing, and Onboarding Engine

The Control Plane lives in proj-control-plane-prod. It is out-of-band — Alice's live inference turns never traverse it, eliminating latency bottlenecks and SPOFs.

Identity & Tenant Lifecycle:

When Tenant A onboarded, the Control Plane provisioned their entry in the tenant catalog (Spanner), enrolled their approved agent assemblies in the Agent Registry, and bound their OIDC identity provider to IAP. The Registry stores A2A-compatible Agent Cards with tier-scoped tool whitelists and version pins. When Alice logs in, the Control Plane has already decided what she is allowed to run.

Policy & Tokenomics:

The Control Plane defines the token quota policies and Model Armor rules that the Compute Plane will enforce. It aggregates per-tenant usage into BigQuery for billing and gross-margin analytics. The schema is designed for ISV finance teams:

sql

```
-- Control Plane billing schema (BigQuery)
CREATE TABLE agent_usage.tokenomics (
  tenant_id STRING,
  tier STRING,
  model_id STRING,
  input_tokens INT64,
  output_tokens INT64,
  session_id STRING,
  timestamp TIMESTAMP
) PARTITION BY DATE(timestamp);
```

Observability Schema:

The Control Plane also owns the OpenTelemetry contract. It defines the mandatory span attributes (tenant_id, agent_id, tool_name, model_id) and operates the Cloud Trace → BigQuery pipeline that feeds both cost attribution and LLM-as-a-Judge evaluation datasets.

What breaks at 2 AM: Registry cache staleness after a tier downgrade; control plane BigQuery export lag causing quota miscalculations; Spanner tenant catalog partition hotspotting during bulk onboarding.

Section 2: Compute Plane — Alice's Execution Path (~600 words)

The Compute Plane is the request hot path. It receives Alice's prompt, enforces control-plane policy, executes the agent loop, and destroys all evidence.

Ingress Enforcement (Tier 1 PEP):

Alice submits "Analyze Q3 revenue." IAP cryptographically verifies her OIDC JWT and strips forged headers (X-Tenant-ID). The Ingress Agent Gateway exchanges her credential for a short-lived On-Behalf-Of (OBO) Intent Token and applies inline Model Armor prompt filters. If injection is detected, the turn is rejected before compute is provisioned — protecting warm-start costs.

Control Plane Query (The PDP Consult):

Before provisioning, the Gateway queries the Control Plane's Agent Registry to confirm finance-agent-v2 is authorized for Tenant A's tier. The root orchestrator then injects an immutable temp:tenant_id = "tenant_a" into the ADK InvocationContext. All child sub-agents inherit this via async contextvars over mTLS. If the Registry is unreachable, the Gateway fails closed — no default-allow.

Python

```
# Immutable tenant context binding at root
session = session_service.create_session(
    app_name="finance-agent",
    user_id="alice@tenant-a.com",
    state={"temp:tenant_id": "tenant_a", "tier": "growth"}
)
```

Ephemeral Runtime & Resource Isolation:

GEAP Agent Runtime provisions an ephemeral sandbox with strict CPU/RAM limits. There is no persistent VM to patch, no cross-tenant memory residue between turns, and no namespace sprawl. Isolation is enforced by destruction, not configuration.

Session, Memory & Context Caching:

The Session Store fetches Alice's chat history via hard key-prefixed lookups (tenant_a:alice_123). The Memory Bank retrieves cross-session knowledge under the same prefix. When calling Gemini, base system prompts and tool schemas are pulled from a shared context cache (amortized across all tenants), while tenant-specific variables are stored under namespaced keys (tenant_a:cache_key). This shared-prefix caching is where 80% Pool margins come from.

Egress Enforcement & Noisy Neighbor Bulkheads (Tier 2 PEP):

Before the model executes, the Egress Gateway checks Tenant A's remaining token bucket (policy defined in Control Plane, enforced in Compute Plane). During execution, the Gateway validates DPoP tokens, inspects intent via Model Armor, and asserts VPC-SC conditional rules. If Tenant B's agent is in an infinite loop, Tenant A's quota is untouched.

What breaks at 2 AM: Async contextvar leakage between concurrent sub-agents; Memory Bank key collision from a missing prefix; sandbox resource exhaustion from recursive sub-agent spawning; token bucket desync between Redis (real-time enforcement) and BigQuery (billing truth).

Section 3: Data Plane — The Governed Tool & Persistence Boundary (~400 words)

The Data Plane is where the agent touches reality. Every tool call and query crosses three mandatory hops.

The Generic 3-Hop Pattern:

MCP Tool Layer: The agent invokes query_financial_db using the OBO Intent Token. The Agent Registry (consulted via Compute Plane) ensures the tool is version-approved for Tenant A's tier.

Knowledge Layer: Natural language is translated into scoped SQL. The OBO token's tenant claim propagates through the call chain.

Persistence Tier: The connection sets SET LOCAL app.current_tenant_id = 'tenant_a' before querying AlloyDB. PostgreSQL RLS guarantees zero rows return if the session variable is unset or mismatched.

sql

-- Fail-closed data isolation

SET LOCAL app.current_tenant_id = 'tenant_a';

CREATE POLICY tenant_isolation ON financial_records

  USING (tenant_id = current_setting('app.current_tenant_id')::TEXT);

Substrate-Agnostic Semantics:

While this example uses AlloyDB RLS, the 3-hop invariant is universal: tenant claim propagation from application session to data-layer policy enforcement. ISVs on Cloud SQL use identical RLS. Spanner uses IAM fine-grained access control with request.tag tenant conditions. Firestore uses path isolation (/tenants/{tenant_id}/documents/). BigQuery uses Row Access Policies. The pattern is the same; only the DDL changes.

MCP Tool Governance:

The Compute Plane Egress Gateway enforces VPC-SC conditional rules using mcp.toolName, mcp.method, and mcp.tool.isReadOnly. Tenant A's agent may call docs_readonly but not send_email. This is not application logic — it is perimeter policy.

What breaks at 2 AM: Missing SET LOCAL in a connection pool reuse; RLS bypass via superuser connections; NL-to-SQL injection that appends OR 1=1 before the RLS predicate is applied; MCP server outage without circuit-breaking.

Section 4: Cross-Plane Observability & The Operational Contract (~300 words)

Observability is not a plane — it is the stitching between planes. Every span carries tenant_id, agent_id, and model_id from Ingress (Compute) through Runtime (Compute) to AlloyDB (Data), with cost attribution aggregated in BigQuery (Control). When a CISO asks "Did Tenant A's agent access Tenant B's data?" you query one trace, not three dashboards.

The Operational Contract:

The Pool tier runs on a single runbook because the Control Plane manages one Registry, one policy set, and one billing schema. The Compute Plane operates one ephemeral runtime fleet with global SLOs. The Data Plane uses one shared-schema AlloyDB instance with RLS. You can onboard 1,000 tenants with the same headcount it takes to maintain 10 Silo snowflakes.

Upgrade Velocity is Global:

When Google releases Gemini 2.6, the Control Plane updates the model allowlist. The Compute Plane's warm-start runtime fleet picks it up simultaneously for all Pool tenants. There are no per-tenant change windows, no project-level IAM reconciliation, and no VPC-SC perimeter drift.

What breaks at 2 AM: Trace cardinality explosion from high-cardinality user_id tags; cross-plane clock skew breaking token validation; Control Plane Registry outage causing Compute Plane fail-closed rejections across all tenants.